

ArchVision

Hybrid AI Floor Plan Analysis Pipeline

"Converting 2D architectural drawings into structured spatial intelligence"

PROJECT TYPE

Full-Stack AI Web Application

DOMAIN

Computer Vision • Architecture • PropTech

DATE

February 2026

TEAM MEMBERS

Mohammed Ayaan

Parth Talsania

Abhishek Rathod

Harshil Darji

Heli Darji

Table of Contents

- 01 Executive Summary
- 02 Problem Statement
- 03 Our Solution — ArchVision
- 04 System Architecture
- 05 Two-Step AI Pipeline — Deep Dive
- 06 Technology Stack
- 07 Backend Architecture
- 08 Frontend Architecture
- 09 Core Data Structures & Schema
- 10 Key Features Showcase
- 11 Wi-Fi Deadzone Mapper — Downstream Application
- 12 Security & Authentication
- 13 Development Workflow
- 14 Challenges & Learnings
- 15 Future Roadmap

01 EXECUTIVE SUMMARY

ArchVision is a hybrid AI-powered web application that transforms 2D architectural floor plan images and PDFs into rich, structured JSON data — complete with room geometries, semantic labels, parsed dimensions, area calculations, and confidence scores.

The system combines **YOLOv8 instance segmentation** for room detection with **dual-path OCR** (EasyOCR + PaddleOCR) for text extraction, wrapped in a modern **React + FastAPI** full-stack application featuring interactive SVG viewers, analytics dashboards, printable spatial reports, and a Wi-Fi signal simulation proof-of-concept.

- **Smart PDF Extraction** — YOLO-validated pipeline filters only real 2D floor plans from multi-page brochure PDFs.
- **Per-Room OCR** — Targeted text extraction within each detected room's bounding box for superior accuracy.
- **Interactive Viewer** — SVG overlay with zoom, pan, colour-coded polygons, and room selection.
- **Analytics Suite** — KPIs, doughnut charts, radar profiles, and ranked bar charts via Recharts.
- **Spatial Report** — Printable audit document with HVAC and LED estimates.
- **Wi-Fi Mapper** — Downstream application using Euclidean distance for signal simulation.
- **OAuth2 Social Login** — Google and GitHub authentication alongside JWT email/password.

02 PROBLEM STATEMENT

Architectural floor plans remain one of the most information-dense documents in the construction and real estate industries. Yet extracting structured, machine-readable data from these drawings is still a largely **manual, error-prone, and time-consuming** process.

The Theoretical Gap

- **No automated room segmentation** — Existing tools cannot reliably detect individual room boundaries from 2D plans.
- **OCR fails on architectural text** — Standard OCR struggles with rotated dimension text, overlapping labels, and mixed fonts common in floor plans.
- **PDF brochures contain noise** — Real estate PDFs mix floor plans with photographs, logos, amenity renders, and marketing pages — naive extraction captures everything.
- **No structured output** — Even when rooms are identified, converting visual data into a standardized, queryable JSON schema is an unsolved integration challenge.
- **Downstream applications blocked** — Without structured spatial data, applications like smart building analytics, HVAC estimation, and signal simulation cannot be automated.

03 OUR SOLUTION — ARCHVISION

ArchVision addresses every gap identified above through a **modular, two-step AI pipeline** wrapped in a production-grade full-stack web application.

Input Formats	PDF brochures, PNG, JPG, BMP, TIFF
AI Models	YOLOv8-Seg (segmentation) + EasyOCR + PaddleOCR (dual-path OCR)
Output	Frontend Schema v1.0.0 JSON — rooms with geometry, labels, dimensions, area, confidence
Web Stack	React 18 + TypeScript + Tailwind (frontend) — FastAPI + SQLite (backend)
Auth	JWT email/password + OAuth2 (Google, GitHub)
Deployment	Monorepo — Vite dev server proxying to FastAPI

04 SYSTEM ARCHITECTURE

High-Level Architecture

The system follows a clean three-tier architecture with a decoupled frontend, API layer, and AI pipeline backend.

Layer	Technology	Responsibility
Presentation	React + Vite + Tailwind	Interactive UI, SVG viewer, analytics dashboards
API Gateway	FastAPI + SQLAlchemy	REST endpoints, JWT/OAuth auth, job management
AI Pipeline	YOLOv8 + OCR Engines	Room detection, text extraction, semantic parsing
PDF Extraction	PyMuPDF + YOLO Validator	Smart floor plan crop extraction from PDFs
Storage	SQLite + File System	User data, analysis jobs, uploads, results

Data Flow

Input (PDF/Image) → **[PDF?]** Smart Extraction (120 DPI scoring → 450 DPI YOLO validation → region detection → crop validation) → **YOLOv8-Seg** room instance detection → **Per-Room OCR** text extraction → **Semantic Parser** label/dimension structuring → **Frontend Schema Builder** JSON output → **Visualization** overlay + interactive viewer

05 TWO-STEP AI PIPELINE — DEEP DIVE

Step 1: Smart PDF Floor Plan Extraction

The **PDFFloorPlanExtractor** is a multi-stage filtering pipeline that ensures only genuine 2D floor plan crops are extracted from brochure PDFs. This is critical because real estate PDFs contain photographs, logos, amenity renders, and marketing content.

- Page Scoring (120 DPI thumbnails)** — Each page is rendered as a low-res thumbnail and scored using text analysis (room keywords like 'bedroom', 'kitchen' vs negative keywords like 'luxury', 'spa') combined with image features (edge ratio, line count, orthogonal line ratio, colourfulness). Minimum score: 0.55.
- YOLO Page Validation (450 DPI)** — Qualifying pages are rendered at full resolution and validated with the YOLO model to confirm room presence.
- Region Detection** — Edge detection + morphological operations + connected components identify up to 4 candidate floor plan regions per page.
- YOLO Crop Validation** — Each candidate crop is validated independently; only crops with ≥3 detected rooms pass through.

Extraction Configuration

DPI (full render)	450
-------------------	-----



Thumbnail DPI	120
Min page score	0.55
Min crop score	0.60
Validator	YOLO
Min rooms per crop	3
Target long side	4096 px
Scan all pages	True

Step 2: Floor Plan Analysis

- **Room Instance Segmentation** — YOLOv8-Seg model (conf=0.35, IoU=0.5, img_size=640) detects rooms with pixel-level segmentation masks. Overlapping detections filtered at IoU > 0.85.
- **Per-Room OCR** — Each room's bounding box (padded by 20px) is cropped and passed to EasyOCR (primary) or PaddleOCR (fallback). Text tokens include confidence and bounding box data.
- **Semantic Parsing** — Extracted text is parsed to identify room labels (e.g., 'Bedroom'), dimension strings (e.g., 12'6" × 10'), and area text (e.g., '125 SQFT'). Dimensions are decomposed into feet + inches with computed area.
- **Geometry Processing** — Raw segmentation masks are converted to Shapely polygons, simplified (tolerance=2.0), and enriched with centroids, bounding boxes, and pixel areas.

-
- **Schema Generation** — All data is assembled into Frontend Schema v1.0.0 JSON for consumption by the web application.

AI & Backend

Technology	Version / Variant	Purpose
Python	3.10+	Core backend language
FastAPI	0.110+	Async REST API framework
YOLOv8-Seg	Ultralytics	Room instance segmentation with masks
EasyOCR	Latest	Primary OCR engine for text extraction
PaddleOCR	Latest	Fallback OCR engine for robustness
PyMuPDF (fitz)	Latest	PDF text extraction and image rendering
OpenCV	Headless	Image processing, edge detection, morphology
Shapely	Latest	Polygon operations and spatial analysis
SQLAlchemy	Latest	ORM for SQLite database access
PyTorch	CPU build	ML backend for YOLO inference
NumPy / Pillow	Latest	Array operations and image I/O

Frontend & Visualization

Technology	Version	Purpose
React	18	Component-based UI framework
TypeScript	5+	Type-safe frontend development
Vite	5	Fast dev server and build tool
Tailwind CSS	3	Utility-first styling
shadcn/ui + Radix	Latest	Accessible UI component primitives
Recharts	Latest	Charts: doughnut, radar, bar, pie
Framer Motion	Latest	Animations: heatmap pulse, transitions
TanStack Query	Latest	Server-state data fetching and caching
Axios	Latest	HTTP client with JWT interceptors
React Router	v6	Client-side routing

07 BACKEND ARCHITECTURE

Directory Structure

- **main.py** — FastAPI app entry point with lifespan, CORS, 4 routers, /api/health
- **config.py** — Paths, secrets, OAuth creds, file limits
- **database.py** — SQLAlchemy engine + SessionLocal + get_db dependency
- **models.py** — ORM: User (email, hashed_password, full_name) and AnalysisJob (status, result_json, metrics)
- **auth.py** — JWT creation + bcrypt hashing + get_current_user dependency
- **routes/auth_routes.py** — POST register, POST login, GET me
- **routes/oauth_routes.py** — Google & GitHub OAuth flows with CSRF state cookies
- **routes/analysis_routes.py** — POST analyze, GET list, GET detail, GET download, DELETE, GET dashboard
- **routes/files_routes.py** — Public file serving (UUID filename security)
- **services/pipeline_service.py** — Singleton AI pipeline wrapper (mirrors Kaggle notebook)

API Endpoints Summary

Endpoint	Method	Auth	Description
/api/auth/register	POST	No	Create account → JWT
/api/auth/login	POST	No	Authenticate → JWT
/api/auth/me	GET	JWT	Current user profile
/api/auth/google	GET	No	Redirect to Google OAuth
/api/auth/github	GET	No	Redirect to GitHub OAuth
/api/analyze	POST	JWT	Upload + analyze file
/api/analyses	GET	JWT	Paginated job list
/api/analyses/{id}	GET	JWT	Job detail with JSON

/api/analyses/{id}/download	GET	JW T	Download result JSON
/api/dashboard	GET	JW T	Aggregated statistics
/api/files/uploads/{name}	GET	No	Serve uploaded file
/api/files/results/{id}/{name}	GET	No	Serve result file
/api/health	GET	No	Health check

08 FRONTEND ARCHITECTURE

The frontend is a single-page React application with a **"Deep Blue Galaxy"** design system — dark glassmorphism with cyan accents, Space Grotesk typography, and rich CSS animations.

Page Routing

Route	Component	Access	Description
/login	Login	Public	Glassmorphic login + OAuth buttons
/register	Register	Public	Registration form
/auth/callback	OAuthCallback	Public	OAuth redirect handler
/dashboard	Dashboard	Protected	Overview + How It Works + Tech Stack
/upload	Upload	Protected	Drag-and-drop + PipelineOverlay animation
/analysis/:id	AnalysisView	Protected	Viewer + Analytics + Report + Wi-Fi
/history	History	Protected	Paginated analysis table
/introduction	ProjectIntro	Protected	Intelligence for Spatial Design
/founders	Founders	Protected	Team bio cards
/wifi-mapper	WifiMapper	Protected	Wi-Fi signal simulation

Key UI Components

- **TelemetryBar** — Fixed top bar with LIVE system status marquee (OpenCV, EasyOCR telemetry).
- **PipelineOverlay** — Full-screen 4-step animated stepper (INGEST → SEGMENT → RECOGNIZE → FUSE) shown during analysis.
- **FloorPlanViewer** — Interactive SVG overlay with zoom (scroll/buttons), pan (Ctrl+click), colour-coded room polygons, hover tooltips, and centroid labels.
- **RoomDetailPanel** — Selected room details: parsed dimensions, area, confidence bars, geometry info.

- **AnalyticsDashboard** — Full Recharts suite: 4 KPI hero cards, doughnut (space allocation), radar (architectural balance), horizontal bar (room hierarchy).
- **SpatialReport** — Portal-rendered printable audit: segmented blueprint, spatial data matrix, HVAC/LED estimates.
- **HowItWorks** — Two-step pipeline explainer with lightbox images and animated data-stream connectors.
- **TechStack** — Custom SVG logo grid with brand-colour hover glows.

09 CORE DATA STRUCTURES & SCHEMA

Frontend Schema v1.0.0 — Output Format

Every analysis produces a JSON document following this standardised schema, enabling consistent consumption by the frontend viewer, analytics, and downstream applications.

Field	Type	Description
schema_version	string	Always '1.0.0'
generated_at	ISO datetime	Timestamp of analysis
source.type	pdf image	Input file type
pages[].rooms[].id	integer	Unique room identifier
rooms[].label	string	Semantic room label (Bedroom, Kitchen, etc.)
rooms[].dimensions	string	Raw dimension text (12'6" x 10')
rooms[].dimensions_parsed	object	Decomposed: width_ft, width_in, height_ft, height_in, area_sqft
rooms[].area.value_sqft	float	Area in square feet
rooms[].area.source	string	computed_from_dimensions ocr_text none
rooms[].confidence	object	geometry, label, dimensions scores (0-1)
rooms[].geometry.polygon	int[][]	Simplified polygon coordinates
rooms[].geometry.centroid	object	{ x, y } centre point
rooms[].geometry.bbox	object	{ x1, y1, x2, y2 } bounding box

Internal Data Classes

- **OCRToken** — text, conf, box (polygon coords), box_poly (Shapely), source ('ocr' | 'pdf_text'), meta dict.
- **RoomInstance** — id, yolo_conf, mask (ndarray), polygon, polygon_simplified, bbox, centroid, area_px2, label, label_confidence, dimensions_text, dimension_parsed, area_text, area_ft2, ocr_tokens, ocr_lines, ocr_merged_text.
- **PipelineConfig** — model_path, yolo_img_size (640), yolo_conf (0.35), yolo_iou (0.5), polygon_simplify_tol (2.0), room_crop_pad (20), ocr_mode ('per_room'), device ('auto'), text_source ('auto').

10 KEY FEATURES SHOWCASE

Smart PDF Extraction

YOLO-validated multi-stage pipeline filters real 2D floor plans from noisy brochure PDFs. Combines text scoring, image feature analysis, and neural network validation to achieve near-zero false positive extraction.

Interactive Floor Plan Viewer

SVG overlay on the original image with colour-coded room polygons, zoom/pan controls, room selection, hover tooltips with dimensions, and centroid labels. Keyboard shortcut (Escape) to deselect.

Dual-Path OCR Engine

EasyOCR as primary engine with PaddleOCR fallback. Per-room cropping with configurable padding ensures text extraction is contextually targeted rather than full-image.

Analytics Dashboard

Four animated KPI cards (total area, room count, largest space, efficiency gauge), doughnut chart for space allocation, radar chart for architectural balance, and horizontal bar chart ranking rooms by area.

Printable Spatial Report

One-click export generates a professional audit document with document ID, segmented blueprint, spatial data matrix, occupancy types, and smart building estimates (HVAC tonnage, LED wattage).

Rendered via React Portal for clean print styling.

Animated Pipeline Overlay

Full-screen 4-step stepper animation (INGEST → SEGMENT → RECOGNIZE → FUSE) with spinning dashed rings, scan-line thumbnail preview, and timed step transitions.

OAuth2 Social Login

Google and GitHub authentication with CSRF state cookies, find-or-create user pattern, and seamless JWT token handoff to the SPA via redirect callback.

11 WI-FI DEADZONE MAPPER — DOWNSTREAM APPLICATION

The Wi-Fi Deadzone Mapper is a **proof-of-concept downstream application** demonstrating that ArchVision's structured output enables real-world spatial analytics beyond simple visualization.

How It Works

- **Data Handoff** — AnalysisView maps pipeline rooms to {id, label, x, y, area} using centroids and saves to localStorage alongside the blueprint image URL.
- **Coordinate Transformation** — Pipeline centroids (original image pixel space) are mapped to the scaled container using CSS object-contain math: $scale = \min(cW/nW, cH/nH)$, with letterbox offset.
- **Signal Calculation** — $MAX_RANGE = container\ diagonal \times 0.85$. Raw signal = $\max(0, 100 - (distance / MAX_RANGE) \times 100)$. Wall penalty: -15% per non-router room crossed.
- **Heatmap Visualization** — Framer Motion pulsing radial gradient (1400px, 3s infinite easeInOut) at router position. Node colours: Cyan ($\geq 80\%$), Amber (40–79%), Rose ($< 40\%$).
- **Analytics Panel** — Average signal metric, Recharts vertical BarChart with conditional cell colours, and signal tier legend.

12 SECURITY & AUTHENTICATION

JWT Tokens	24-hour expiration, HS256 algorithm, stored in localStorage
Password Hashing	bcrypt (direct, not passlib) for all user passwords
OAuth2 CSRF	State parameter in signed cookie, verified on callback
File Upload Validation	Type whitelist (PNG, JPG, PDF, BMP, TIFF), 50 MB max
Path Traversal Protection	Filename sanitisation on file-serving endpoints
CORS Policy	Configured for specific origins (localhost:8080, 5173)
File Serving Security	Public endpoints secured via UUID-prefixed filenames
OAuth User Creation	Random bcrypt hash for OAuth-created users (no reusable password)
API Authorization	Bearer token required on all /api/analyze, /api/analyses, /api/dashboard

13 DEVELOPMENT WORKFLOW

Running the Backend

```
python -m uvicorn backend.main:app --reload --port 8000
```

API documentation available at <http://127.0.0.1:8000/docs> (Swagger UI).

Running the Frontend

```
cd archvision-login-glow-main && npm run dev
```

Dev server at <http://localhost:8080> with Vite proxy forwarding /api → backend.

Environment Variables

ARCHVISION_SECRET_KEY	JWT signing secret
ARCHVISION_MODEL_PATH	Path to YOLOv8 best.pt weights
GOOGLE_CLIENT_ID / SECRET	Google OAuth credentials
GITHUB_CLIENT_ID / SECRET	GitHub OAuth credentials
ARCHVISION_FRONTEND_URL	http://localhost:8080
ARCHVISION_BACKEND_URL	http://localhost:8000

14 CHALLENGES & LEARNINGS

PDF Noise Filtering

Real estate brochure PDFs contain photographs, amenity renders, logos, and marketing pages alongside actual floor plans. Naive extraction produced 80%+ false positives. Solution: multi-stage scoring (text + image features) combined with YOLO validation at both page and crop levels reduced false positives to near zero.

OCR Accuracy on Architectural Text

Dimension strings (12'6" × 10') use special characters and mixed fonts that confuse standard OCR. Solution: per-room cropping with padding isolates text context, and dual-engine fallback (EasyOCR → PaddleOCR) maximises extraction recall.

Coordinate Space Mapping

Pipeline outputs centroids in original image pixel coordinates, but the frontend renders images with CSS object-contain scaling. Misaligned coordinates caused Wi-Fi mapper nodes to appear in wrong positions. Solution: compute scale + letterbox offset to transform coordinates.

Large PDF Processing Time

Multi-page PDFs at 450 DPI with YOLO inference per page/crop created long processing times. Solution: 120 DPI thumbnail pre-screening eliminates non-candidate pages before expensive full-resolution rendering, and a 10-minute timeout with animated PipelineOverlay keeps users informed.

OAuth Integration Complexity

Handling CSRF state, cookie signing, token exchange, and find-or-create user patterns across Google and GitHub required careful async HTTP flows. Solution: `httpx.AsyncClient` with state cookies and unified JWT handoff via redirect callback.

15 FUTURE ROADMAP

- **3D Model Generation** — Extrude detected room polygons into 3D meshes for virtual walkthroughs using Three.js or Blender integration.
- **Multi-Model Ensemble** — Combine YOLOv8 with Mask R-CNN and SAM (Segment Anything Model) for improved segmentation on complex plans.
- **Dimension Line Detection** — Train a dedicated model to detect dimension line annotations (arrows, tick marks) for more accurate measurement extraction.
- **Real-Time Collaboration** — WebSocket-based multi-user editing of floor plan annotations with conflict resolution.
- **Cloud Deployment** — Dockerised deployment on AWS/GCP with GPU-accelerated inference, S3 storage, and PostgreSQL.
- **Mobile Application** — React Native companion app for on-site floor plan capture and instant analysis.
- **Custom YOLO Training UI** — In-app interface for users to annotate and fine-tune the YOLO model on their specific floor plan styles.
- **HVAC & Electrical Layout** — Extend the pipeline to detect and place HVAC ducts, electrical outlets, and plumbing fixtures.

16 TEAM

ArchVision was built by a team of five students from the **Institute of Advanced Research (IAR)** as part of their 6th semester project.

Name	Role	Contribution
Parth Talsania	Project Lead , Full-Stack	Architecture design, AI pipeline, backend API, frontend UI, integration
Mohammed Ayaan	AI / ML Engineer	YOLO model training, segmentation pipeline, OCR integration
Abhishek Rathod	Backend Developer	FastAPI endpoints, database models, authentication system
Harshil Darji	Frontend Developer	React components, interactive viewer, analytics dashboard
Heli Darji	UI/UX & Testing	Design system, user experience, quality assurance

Thank you for reviewing the ArchVision project report. This document provides a comprehensive technical overview of our hybrid AI floor plan analysis pipeline and its full-stack web application.